

8.3 The Matching Problem

8.4 The Adwords Problem

Slides Notes

by

Zahid Ur Rahman (PhD Course IE&CE)

8.3.1 - Matches and Perfect Matches

- A bipartite graph is a special type of graph that has two distinct sets of nodes, with edges only connecting nodes from one set to nodes in the other set. In this example:
- Left set: Nodes labeled 1, 2, 3, and 4 Right set: Nodes labeled a, b, c, and d
- The key characteristics shown in the diagram:
 1. No edges connect nodes within the same set (e.g., 1 doesn't connect to 2, and a doesn't connect to b)
 2. Edges only go between the left and right sets
 3. Each line represents a potential match/connection between nodes

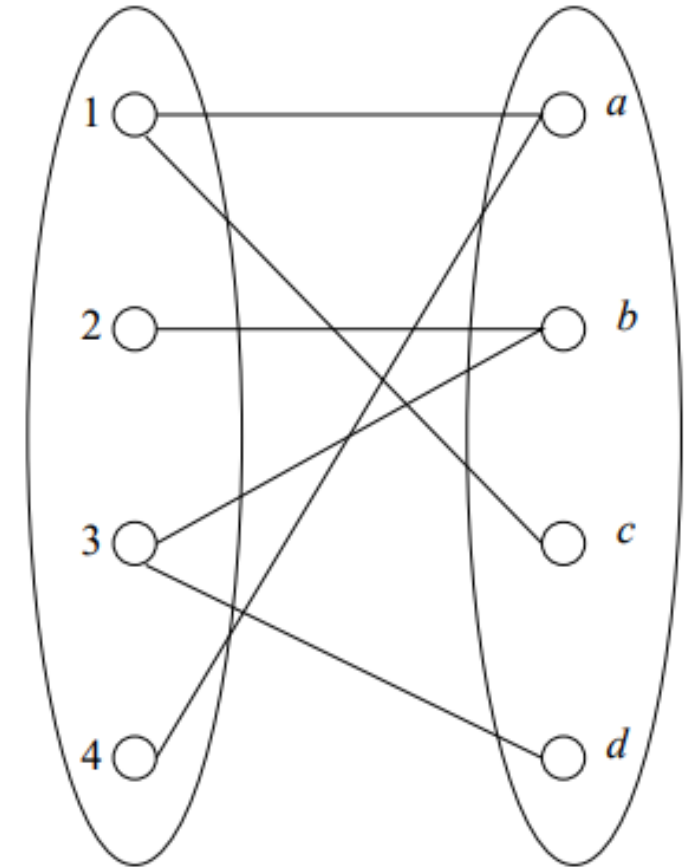


Figure 8.1: A bipartite graph

Matches and Perfect Matches

Matching is a subset of the edges

- Suppose we are given a bipartite graph. A matching is a subset of the edges such that no node is an end of two or more edges.
- The set of edges
- $\{(1, a), (2, b), (3, d)\}$
- is a matching for the bipartite graph

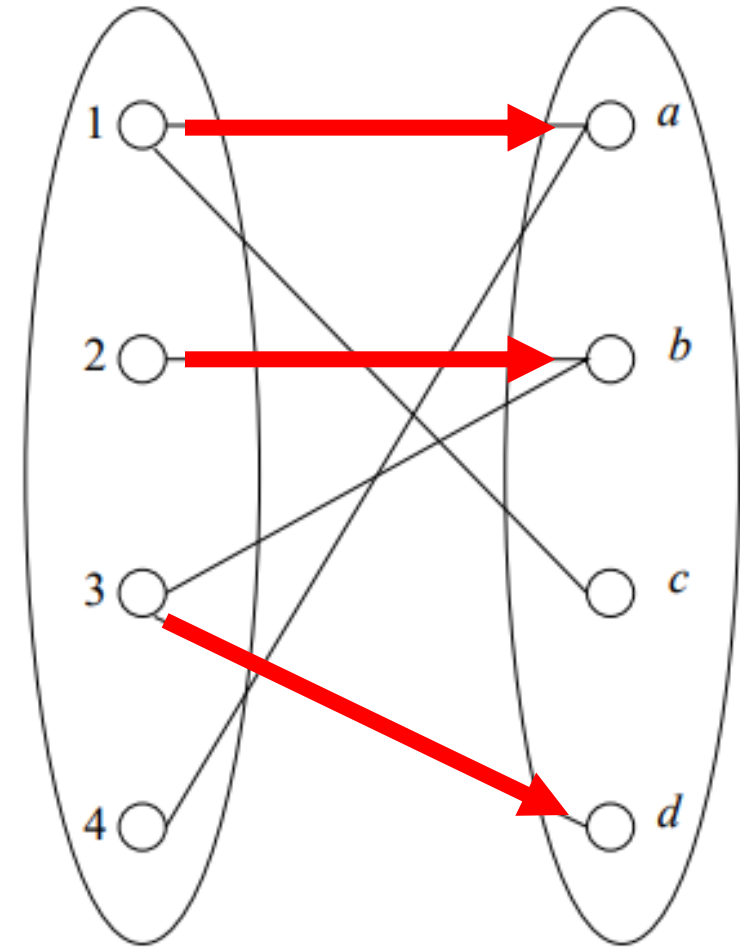


Figure 8.1: A bipartite graph

Matches and Perfect Matches

Perfect Matching:

- A matching is said to be perfect if every node appears in the matching.
- Note that a matching can only be perfect if the left and right sets are of the same size.
- The set of edges
- $\{(1, c), (2, b), (3, d), (4, a)\}$
- Are perfect matching for the bipartite graph

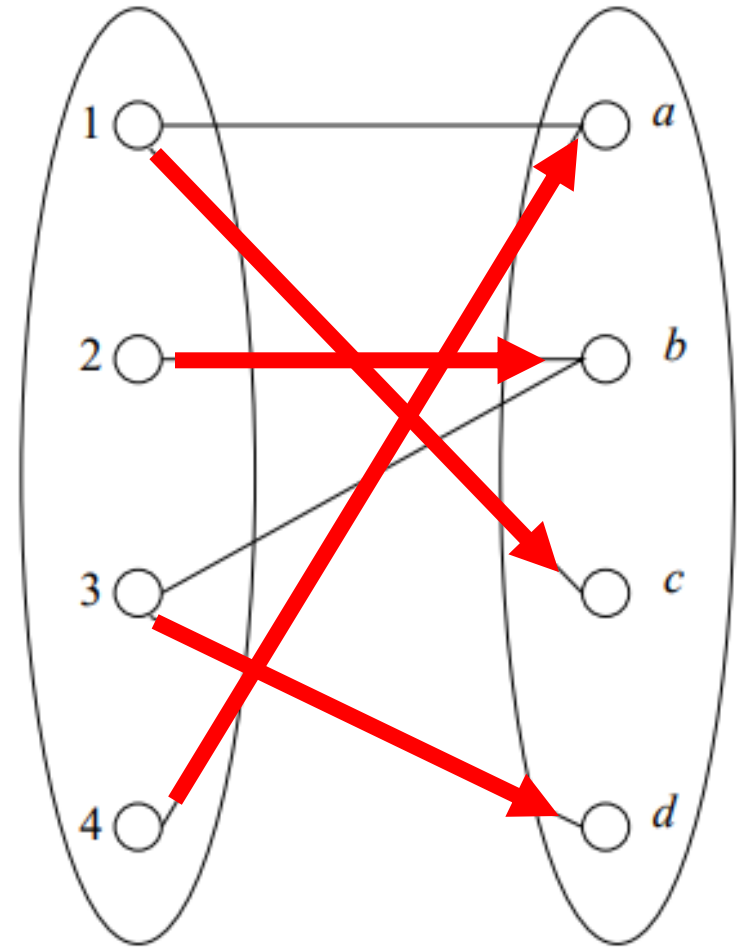


Figure 8.1: A bipartite graph

Matches and Perfect Matches

Maximal Matching:

- A matching that is as large as any other matching for the graph in question is said to be maximal.
- since every perfect matching is maximal.
- The set of edges
- $\{(1, c), (2, b), (3, d), (4, a)\}$
- Are perfect matching as well as maximal for the bipartite graph

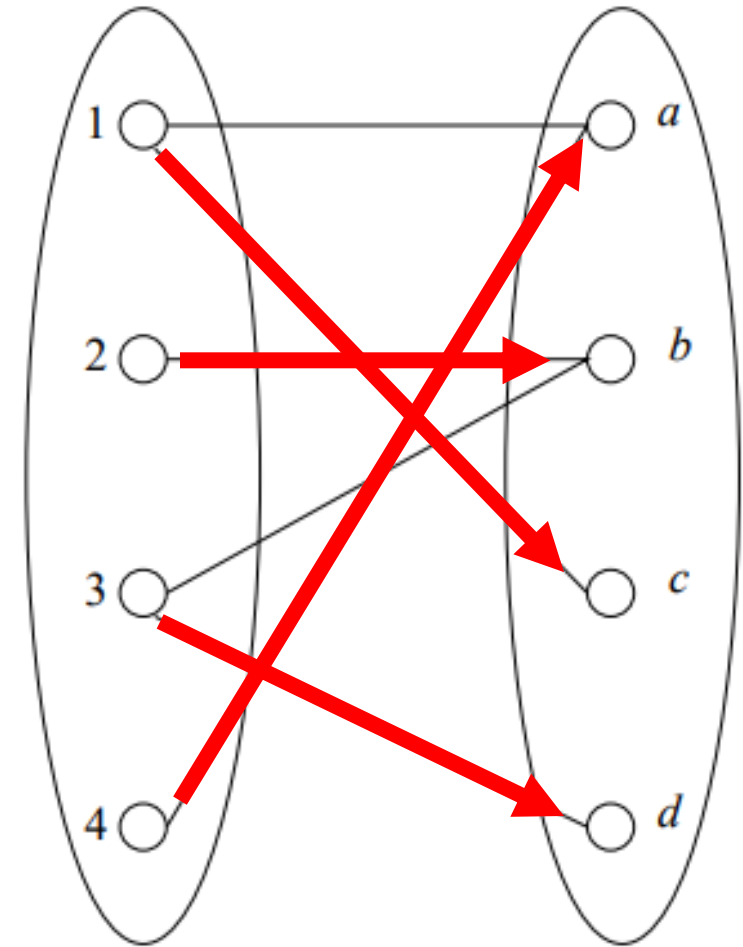


Figure 8.1: A bipartite graph

8.3.2 - (Greedy Match)

- **Example 8.5:** Let us consider a greedy match for the graph of Fig. 8.1
- Suppose we order the nodes lexicographically, that is, by order of their left node, breaking ties by the right node.
- Then we consider the edges in the order
- $(1, a), (1, c), (2, b), (3, b), (3, d), (4, a)$.
- The first edge, $(1, a)$, surely becomes part of the matching. The second edge, $(1, c)$, cannot be chosen, because node 1 already appears in the matching. The third edge, $(2, b)$, is selected, because neither node 2 nor node b appears in the matching so far. Edge $(3, b)$ is rejected for the match because b is already matched, but then $(3, d)$ is added to the match because neither 3 nor d has been matched so far. Finally, $(4, a)$ is rejected because a appears in the match.
- Thus, the matching produced by the greedy algorithm for this ordering of the edges is $\{(1, a), (2, b), (3, d)\}$.
- As we saw, this matching is not maximal.

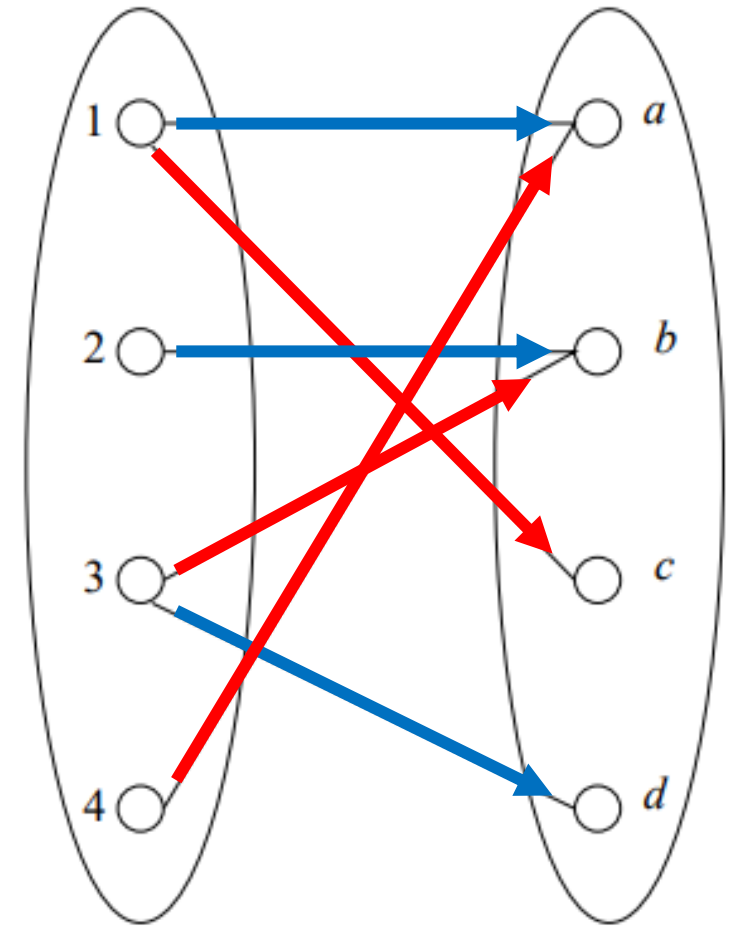


Figure 8.1: A bipartite graph

Example 8.6 (Greedy Match)

- Example 8.6 : A greedy match can be even worse than that of Example 8.5.
- On the graph of Fig. 8.1, any ordering that begins with the two edges $(1, a)$ and $(3, b)$, in either order, will match those two pairs but then will be unable to match nodes 2 or 4. Thus, the size of the resulting match is only 2.

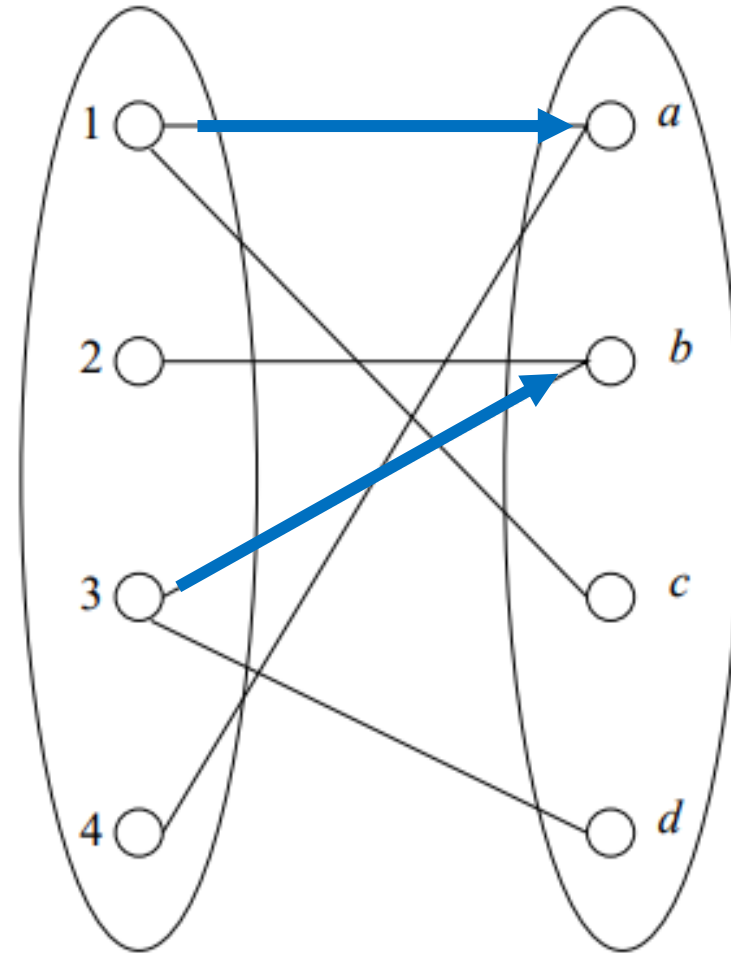


Figure 8.1: A bipartite graph

8.3.3 - Competitive Ratio for Greedy Matching

- We're comparing two ways of matching:
 - Greedy Algorithm (M_g): Makes immediate matches as edges appear.
 - Optimal Matching (M_o): Best possible matching if we knew all edges in advance.

2.The Main Claim:

- Greedy will always get at least half as many matches as the optimal solution.
- So if optimal can match 100 pairs, greedy will match at least 50.

8.4 - The Adwords Problem:

- Around the year 2000, a company called Overture (later bought by Yahoo!) introduced a new kind of search.
- Advertisers bid on keywords (words in a search query), and when a user searched for that keyword, the links to all the advertisers who bid on that keyword are displayed.
- If the advertiser's link was clicked on, they paid what they had bid.
- That sort of search was very useful for the case where the search queryer really was looking for advertisements, but it was rather useless for the queryer who was just looking for information.

8.4.1 - The Adwords Problem (History):

- Several years later, Google adapted the idea in a system called Adwords.
- Google would show only a limited number of ads with each query by deciding which ads to show, as well as the order in which to show them.
- Users of the Adwords system specified a budget: the amount they were willing to pay for all clicks on their ads in a month. These constraints make the problem of assigning ads to search queries more complex, as we hinted at in Example 8.1.
- Google did not simply order ads by the amount of the bid, but by the amount they expected to receive for display of each ad. That is, the clickthrough rate was observed for each ad, based on the history of displays of that ad. The value of an ad was taken to be the product of the bid and the click-through rate.

(8.4.2) - Definition of the Adwords Problem:

Given:

- A set of bids by advertisers for search queries.
- A click-through rate for each advertiser-query pair.
- A budget for each advertiser. We shall assume budgets are for a month, although any unit of time could be used.
- A limit on the number of ads to be displayed with each search query.

Respond to each search query with a set of advertisers such that:

- The size of the set is no larger than the limit on the number of ads per query.
- Each advertiser has bid on the search query.
- Each advertiser has enough budget left to pay for the ad if it is clicked upon.

(8.4.3) - The Greedy Approach to the Adwords Problem:

1. The Problem Setup:

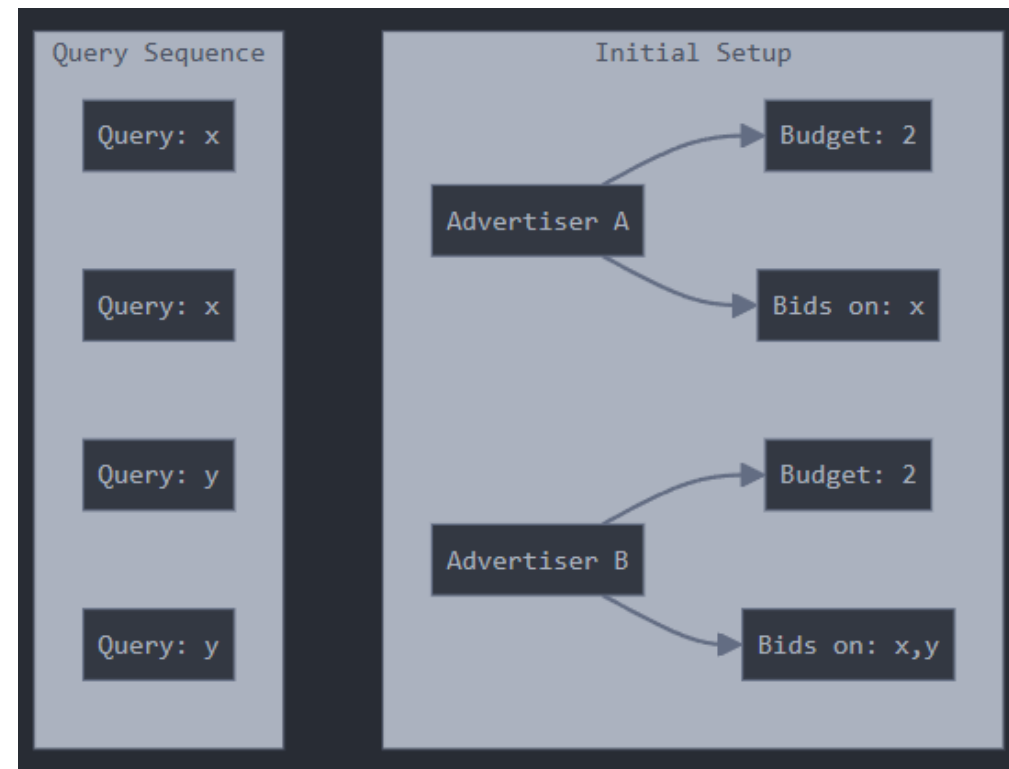
- It's simplified to have these constraints:
 - Only one ad is shown per query
 - All advertisers have equal budgets
 - Click-through rates are identical
 - Bids can only be 0 or 1 (binary)

2. The Greedy Algorithm:

- For each search query, it simply picks any advertiser who has bid 1 for that query
- It makes immediate decisions without considering future queries

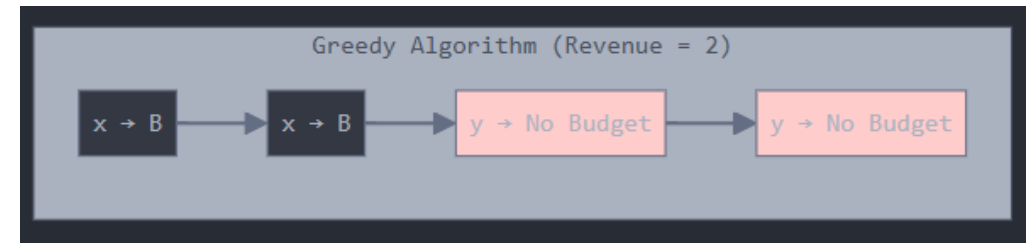
3. The Example Given:

- Two advertisers: A and B
- Two possible queries: x and y
- Advertiser A bids only on query x
- Advertiser B bids on both x and y
- Both advertisers have a budget of 2
- Query sequence: xxyy

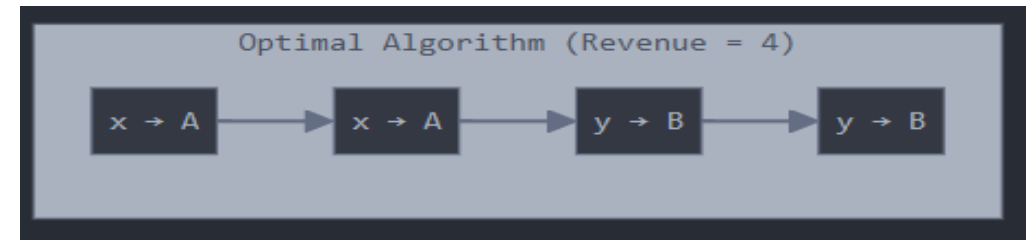


The Greedy Approach to the Adwords Problem

- Greedy Algorithm Performance: Assigns both x queries to B (using up B's budget)
- Can't allocate the y queries (B's budget is exhausted)
- Total revenue = 2



- Optimal Algorithm Performance: Assigns x queries to A
- Assigns y queries to B
- Total revenue = 4



(8.4.4) - The Balance Algorithm

There is a simple improvement to the greedy algorithm that gives a competitive ratio of $3/4$ for the simple case of Section 8.4.3 called the Balance Algorithm.

First Query (x): Both A and B can bid on x

- Both have same budget (2)
- Balance Algorithm can pick either one (let's say it picks A)
- After this: A has 1 budget left, B still has 2

Second Query (x):

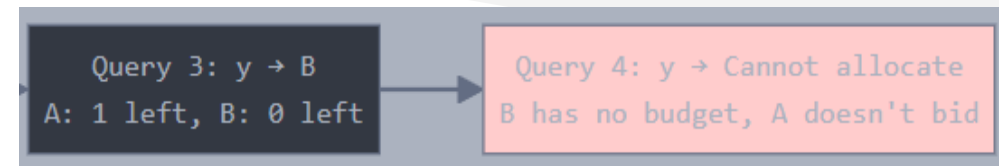
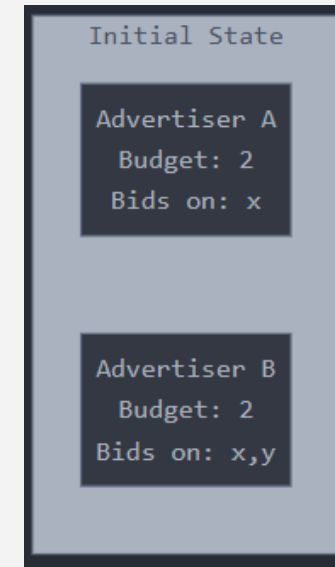
- Must go to B because B has more remaining budget (2) than A (1)
- After this: A has 1 left, B has 1 left

Third Query (y):

- Only B can bid on y
- B has 1 budget left, so gets this query
- After this: A has 1 left, B has 0 left

Fourth Query (y): Cannot be allocated because:

- B has no budget left
- A doesn't bid on y



(8.4.5) A Lower Bound on Competitive Ratio for Balance

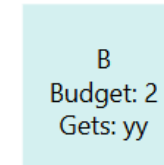
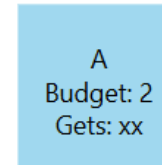
The problem:

- Two advertisers (A_1 and A_2) with same budget B
- Need to assign ads to make most money
- Don't know future queries when making decisions

2. Two Approaches shown in diagram:

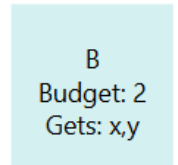
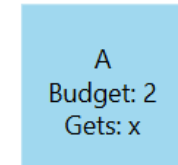
- (a) Optimum (knows future, gets $2B$ revenue):
- Perfectly splits queries between A_1 and A_2
- Each gets B worth of ads
- Total = $2B$ (perfect outcome)
- (b) Balance Algorithm (doesn't know future, gets $3B/4$ revenue):
- Always tries to balance remaining budgets
- Ends up with:
 - A_2 gets fully used (B)
 - A_1 gets partially used (y)
 - Some queries (x) can't be used
 - But proves y is always $\geq x$

Optimal Algorithm (Knows Future)



Revenue = 4

Balance Algorithm (Real-time Decisions)



Revenue = 3

Query Sequence: $x \rightarrow x \rightarrow y \rightarrow y$

Competitive Ratio = $3/4$ (Balance achieves at least 75% of Optimal)

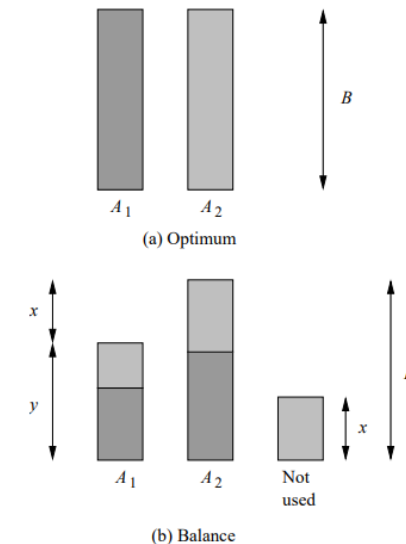


Figure 8.3: Illustration of the assignments of queries to advertisers in the optimum and Balance algorithms

(8.4.6) - The Balance Algorithm with Many Bidders

- When there are many advertisers, the competitive ratio for the Balance Algorithm can be under $3/4$, but not too far below that fraction. The worst case for Balance is as follows.

3. Setup Terms

- N advertisers: Labeled as $A_1, A_2, \dots, A_n$
- $B = N!$ (factorial): The budget each advertiser has
- N queries: Labeled as $q_1, q_2, \dots, q_n$
- Round: A sequence of B identical queries

4. Bidding Pattern

Let's make this very clear with an example where $N = 4$:

- A_1 bids on: only q_1
- A_2 bids on: q_1 and q_2
- A_3 bids on: q_1, q_2 , and q_3
- A_4 bids on: q_1, q_2, q_3 , and q_4

5. Query Sequence

If $B = 24$ ($4!$), the sequence would be:

- Round 1: 24 copies of q_1
- Round 2: 24 copies of q_2
- Round 3: 24 copies of q_3
- Round 4: 24 copies of q_4

6. Distribution Pattern

Let's track how queries get distributed in each round with $N = 4$:

Round 1 (q_1):

- All 4 advertisers bid
- Each gets $B/4 = 6$ queries
- All budgets reduced by 6

Round 2 (q_2):

- Only A_2, A_3, A_4 bid (A_1 doesn't bid on q_2)
- Each gets $B/3 = 8$ queries
- Their budgets further reduced

Round 3 (q_3):

- Only A_3, A_4 bid
- Each gets $B/2 = 12$ queries
- Budgets might be exhausted here

The Balance Algorithm with Many Bidders – (Continue)

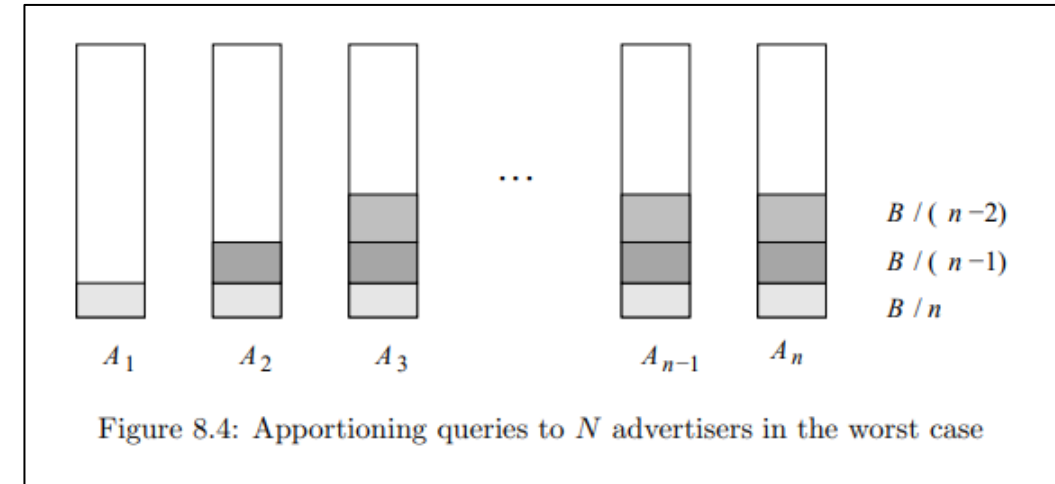
1. Vertical Columns: Each column represents an advertiser (A_1 through A_n)

2. Shaded Sections: The shaded sections in each column show how queries are distributed:

1. Bottom section (lightest): B/N queries from round 1
2. Middle section: $B/(N-1)$ queries from round 2
3. Top section (darkest): $B/(N-2)$ queries from round 3, and so on

3. Distribution Pattern:

1. A_1 gets only B/N queries (from round 1)
2. A_2 gets B/N queries from round 1 + $B/(N-1)$ from round 2
3. A_3 gets B/N from round 1 + $B/(N-1)$ from round 2 + $B/(N-2)$ from round 3
1. Pattern continues for all advertisers



4. Key Observations:

1. Later advertisers (right side) participate in more rounds
 2. Each round's queries are split among fewer advertisers
 3. The fractions get larger in later rounds ($B/(N-1) > B/N$)
 4. This shows why budgets get exhausted before all rounds complete
- This distribution pattern is what leads to the suboptimal performance of the Balance Algorithm, achieving only about 63.2% ($1-1/e$) of the optimal revenue in the worst case.

$$B\left(\frac{1}{N} + \frac{1}{N-1} + \cdots + \frac{1}{N-j+1}\right) \geq B$$

The Balance Algorithm with Many Bidders – (Continue)

1. First, it references Euler's observation that when k becomes large, the sum of reciprocals $(1/1 + 1/2 + 1/3 + \dots + 1/k)$ approaches the natural logarithm of k .
2. Using this observation, they're looking at an equation where they want: $\log(N) - \log(N-j) = 1$
3. Through the properties of logarithms, $\log(N) - \log(N-j)$ can be rewritten as $\log(N/(N-j))$
4. So the equation becomes: $\log(N/(N-j)) = 1$
5. When you exponentiate both sides (using e as the base), you get: $N/(N-j) = e$
6. Solving this equation for j yields: $j = N(1 - 1/e)$

(8.4.7) - The Generalized Balance Algorithm

- The Balance Algorithm works well when all bids are 0 or 1. However, in practice, bids can be arbitrary, and with arbitrary bids and budgets Balance fails to weight the sizes of the bids properly. The following example illustrates this point:

- **Example 8.9 (Problem with Original Balance Algorithm)**

- Setup:

- 2 Advertisers: A₁ and A₂
- 1 query 'q' appears 10 times
- A₁: Bids \$1, Budget \$110
- A₂: Bids \$10, Budget \$100
- What happens:

- 1. **Optimal Solution:**

1. Give all 10 queries to A₂
2. Revenue = 10 queries × \$10 = \$100

- 2. **Balance Algorithm's Poor Choice:**

1. Gives all queries to A₁ (because A₁ has larger budget: \$110 > \$100)
2. Revenue = 10 queries × \$1 = \$10
3. This is much worse! Only gets \$10 instead of possible \$100

(8.4.7) - The Generalized Balance Algorithm

- **Example 8.10 (Improved Balance Algorithm)**

- Same setup, but now uses new formula: $\Psi = \text{bid} \times (1 - e^{(-\text{fraction_of_budget_remaining})})$
- For first query:

1. **A₁'s score:**

1. Bid = \$1
2. Full budget remains (fraction = 1)
3. $\Psi_1 = 1 \times (1 - 1/e) = 0.63$

2. **A₂'s score:**

1. Bid = \$10
2. Full budget remains (fraction = 1)
3. $\Psi_2 = 10 \times (1 - 1/e) = 6.3$

- Result:
- A₂ wins because $6.3 > 0.63$
- Even after using 90% of A₂'s budget, their score stays above A₁'s
- All queries correctly go to A₂
- Gets optimal revenue of \$100
- The improved algorithm works better because it:
 1. Considers bid amounts (fixes ignorance of high bids)
 2. Still looks at remaining budget (prevents overuse)
 3. Makes smarter decisions by balancing both factors

Thank You

